

LAB05

Danillo Lemes

Leitura dos pacotes

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2     4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr       1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(ggplot2)
library(ggcal)
library(leaflet)
```

Warning: pacote 'leaflet' foi compilado no R versão 4.6.1

```
library(geosphere) # Pacote para calcular distancias na Terra
```

Warning: pacote 'geosphere' foi compilado no R versão 4.6.1

Parte 1: Crie uma função que: a. Receba um valor de TAIL_NUMBER (por exemplo, N431WN); b. Produza uma tabela (tidy) com todos os trajetos realizados pela aeronave (ordenadas por data e hora, contendo todas as colunas do arquivo flights.csv.zip); c. Produza

um mapa que apresente todo o trajeto voado pela aeronave ao longo de todo o ano; o trajeto deve ser apresentado de maneira linear no tempo (i.e., segue a sequência do tempo, como no exemplo hipotético dado acima); d. O mapa deve ser decorado com estatísticas do seu interesse (por exemplo, velocidade média do voo como espessura da linha que conecta os aeroportos envolvidos no trajeto);

```
#Criando a função analisa_aeronave, para analisar as aeronaves no banco de dados
analisa_aeronave <- function(tail_number, arquivo_flights) {
  #Criando filtros pelo tail_number, para filtrar cada bloco do banco de dados seguinte
  filter_tail_number <- function(tail, pos){
    tail %>%
      filter(TAIL_NUMBER == tail_number) #Mantem as linhas apenas da aeronave informada
  }
  #Leitura do banco de dados flights em formato chunked, devido ao tamanho do banco e chamando
  dados_flights <- read_csv_chunked(arquivo_flights, callback = DataframeCallback$new(filter_tail_number))

  #Verificação se o aeronave não existir o código encerrar e assim não gera sobrecarga do código
  if (nrow(dados_flights) == 0){
    stop(paste("Nenhum voo encontrado para:", tail_number))
  }

  #Leitura do banco dos aeroportos, com colunas selecionadas
  aeroportos <- read_csv("airports.csv", col_select = c("IATA_CODE", "CITY", "STATE", "LATITUDE", "LONGITUDE"),
    filter(!is.na(LATITUDE) & !is.na(LONGITUDE))

  # Deixando a tabela no formato tidy.
  tabela_tidy <- dados_flights %>%
    mutate(
      DEPARTURE_TIME = sprintf("%04d", as.integer(DEPARTURE_TIME)),
      DATETIME = suppressWarnings(
        ymd_hm(paste0(YEAR, "-", MONTH, "-", DAY, " ",
          substr(DEPARTURE_TIME, 1, 2), ":",
          substr(DEPARTURE_TIME, 3, 4)))
      )
    ) %>%
  #Ordenando o banco pela data e hora do voo
  arrange(DATETIME, YEAR, MONTH, DAY, DEPARTURE_TIME) %>%
  inner_join(
    aeroportos %>% rename_with(~ paste0(., "_ORIGIN")),
    by = c("ORIGIN_AIRPORT" = "IATA_CODE_ORIGIN")
  ) %>%
  inner_join(
    aeroportos %>% rename_with(~ paste0(., "_DEST")),
```

```

    by = c("DESTINATION_AIRPORT" = "IATA_CODE_DEST")
      ) %>%
mutate(
  Ordem_Voo = row_number(),
  Media_velocidade = ifelse(!is.na(AIR_TIME) & AIR_TIME > 0,
                             (DISTANCE / AIR_TIME) * 60, NA),
  LABEL_TEXT = paste0(
    "<b>Voo nº: </b>", Ordem_Voo, "<br>",
    "<b>Data: </b>", DAY, "/", MONTH, "/", YEAR, "<br>",
    "<b>Trecho: </b>", ORIGIN_AIRPORT, " → ", DESTINATION_AIRPORT, "<br>",
    "<b>Velocidade Média: </b>", round(Media_velocidade, 1), " mph<br>",
    "<b>Distância: </b>", DISTANCE, " milhas"
  )
)

#Construção do mapa interativo
aeroportos_visitados <- bind_rows( #Consolida em apenas uma lista unica todas as origens e d
  tabela_tidy %>% select(IATA_CODE = ORIGIN_AIRPORT, CITY = CITY_ORIGIN, LAT = LATITUDE_ORIG
  tabela_tidy %>% select(IATA_CODE = DESTINATION_AIRPORT, CITY = CITY_DEST, LAT = LATITUDE_D
) %>%
distinct(IATA_CODE, .keep_all = TRUE)

## Criação do mapa
mapa <- leaflet() %>%
  addProviderTiles(providers$CartoDB.Positron) %>%
  addCircleMarkers(
    data = aeroportos_visitados,
    lng = ~LNG, lat = ~LAT,
    radius = 5,
    color = "#1F77B4",
    fillOpacity = 0.8,
    label = ~paste0(IATA_CODE, " - ", CITY)
  )

# Dentro do laço for na função analisa_aeronave:
for (i in seq_len(nrow(tabela_tidy))) {
  row <- tabela_tidy[i, ]

  # Calculo dos pontos intermedios ao longo da terra
  gc_linha <- gcIntermediate(
    p1 = c(row$LONGITUDE_ORIGIN, row$LATITUDE_ORIGIN),
    p2 = c(row$LONGITUDE_DEST, row$LATITUDE_DEST),

```

```

    n = 50,
    addStartEnd = TRUE
  )

  # Define a espessura proporcional à velocidade
  espessura_linha <- ifelse(!is.na(row$Media_velocidade), (row$Media_velocidade / 200), 0.5)

  #Desenha o segmento de reta do trajeto no mapa
  mapa <- mapa %>%
    addPolylines(
      lng = gc_linha[, 1],
      lat = gc_linha[, 2],
      weight = espessura_linha,
      color = "#FFDBBB",
      opacity = 0.6,
      popup = htmltools::HTML(row$LABEL_TEXT), # Renderiza o HTML no clique
      label = htmltools::HTML(row$LABEL_TEXT), # Destaca ao passar o mouse
      highlightOptions = highlightOptions(
        weight = espessura_linha + 1.5,
        color = "blue",
        opacity = 1.0,
        bringToFront = TRUE # Exibe ao passar o mouse
      )
    )
  }
#Retorna o objeto e a tabela de forma limpa
return(list(
  tabela = tabela_tidy,
  grafico = mapa
))
}

```

#Área de teste

```

#Chama o banco de dados e a função criada
teste <- analisa_aeronave(tail_number = "N431WN", arquivo_flights = "flights.csv.zip")
# Diferentes tail_number's para testar: N028AA, N488WN, N17126, N587JB

#Exibe as primeiras linhas da tabela
head(teste$tabela)

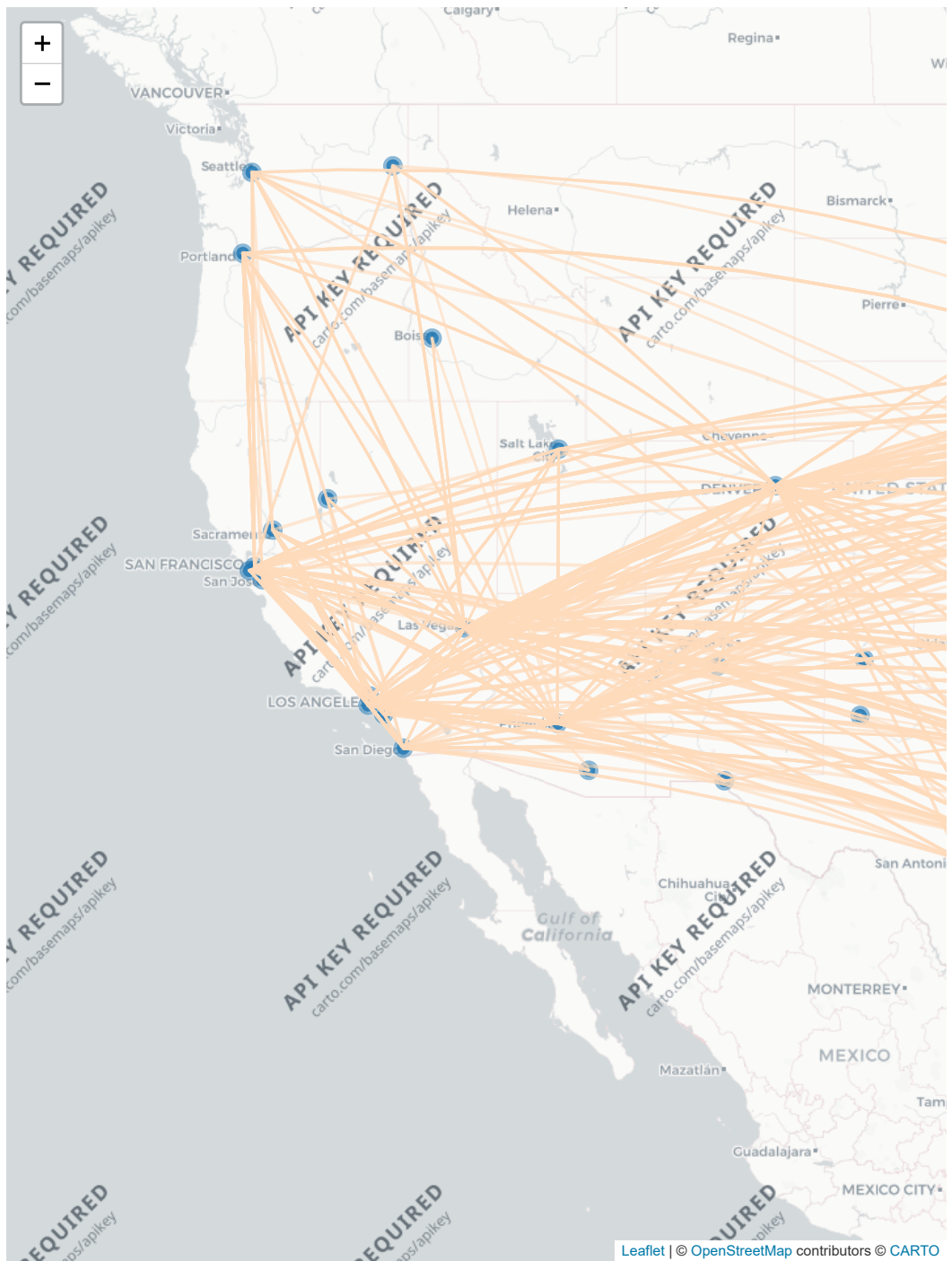
```

A tibble: 6 x 43

	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER	ORIGIN_AIRPORT
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<chr>
1	2015	1	1	4	WN	1762	N431WN	DCA
2	2015	1	1	4	WN	3564	N431WN	HOU
3	2015	1	1	4	WN	3564	N431WN	BOS
4	2015	1	1	4	WN	3564	N431WN	BWI
5	2015	1	2	5	WN	1793	N431WN	MCI
6	2015	1	2	5	WN	1793	N431WN	LAX

i 35 more variables: DESTINATION_AIRPORT <chr>, SCHEDULED_DEPARTURE <chr>,
DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>, TAXI_OUT <dbl>,
WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>, AIR_TIME <dbl>,
DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>, SCHEDULED_ARRIVAL <chr>,
ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>, DIVERTED <dbl>, CANCELLED <dbl>,
CANCELLATION_REASON <chr>, AIR_SYSTEM_DELAY <dbl>, SECURITY_DELAY <dbl>,
AIRLINE_DELAY <dbl>, LATE_AIRCRAFT_DELAY <dbl>, WEATHER_DELAY <dbl>, ...

```
# Renderiza o mapa
teste$grafico
```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 11:50:48 -03"
```